Resumo executivo: Fases 2, 3 e 4

O que foi implementado

Este documento resume a camada de servicos, a ampliacao dos testes automatizados e a refatoracao modular das sete paginas mais extensas do Neuro Dose Assist. Inclui commits, evidencias de verificacao, exemplos de codigo e limitacoes conhecidas.

Visao geral

- **Fase 2 ? Servicos:** contratos TypeScript e implementacoes Firebase foram organizados em uma camada de servicos; as funcoes de dominio delegam para essa camada.
- **Fase 3 ? Testes:** foram adicionados testes de componentes, funcoes de dominio e servicos; tambem foram configurados scripts de cobertura/build e CI.
- **Fase 4 ? Refatoracao:** Appointments, Medications, Clinic, Patients, Settings, Reports e Analytics foram divididas em paginas de composicao, hooks e componentes dedicados.

Estado do worktree e escopo

Branch de trabalho: gabriellmaia01-fase-3-t3-5. Os commits listados neste relatorio estao no historico local; nao houve push. No momento da geracao, o worktree estava limpo. A verificacao visual das rotas protegidas continua pendente porque nao havia configuracao Firebase local valida nem emulador/dados de teste.

Nota de contexto: a Fase 4 foi executada e commitada antes da instrucao posterior para interromper o trabalho. Este relatorio documenta o estado existente; nao adiciona alteracoes ao codigo do produto.

Fase 2 ? Camada de servicos Firebase

Objetivo e organizacao

O acesso a dados foi concentrado em implementacoes de servico e contratos TypeScript. Os contratos descrevem as operacoes utilizadas por cada dominio; a implementacao Firebase cumpre esses contratos. A camada lib mantem funcoes de fachada para os consumidores da aplicacao.

- **Contratos:** src/services/contracts/domain.types.ts declara interfaces como IPatientService, IMedicationService, IDoseService, IAppointmentService, IClinicService e IUserSettingsService.
- **Implementacoes:** src/services/firebase/ contem os servicos Firebase, incluindo patientService, medicationService, doseService, appointmentService, clinicService, noteService, activityLogService e userSettingsService.
- **Fachadas:** src/lib/patients.ts, por exemplo, encaminha operacoes ao servico correspondente, reduzindo acoplamento direto dos consumidores.

Exemplo de contrato

```
export interface IPatientService {
  getPatients(userId: string): Promise<Patient[]>;
  getPatientById(id: string, userId: string): Promise<Patient | null>;
  savePatient(
    patient: Patient, userId: string, userInfo?: UserAuditInfo
  ): Promise<string>;
  deletePatient(
    id: string, userId: string, patientName?: string
  ): Promise<void>;
}
```

O contrato deixa explicita a superficie de operacoes de pacientes e seus tipos de entrada/saida. Isso facilita testar chamadas e manter o limite entre o dominio e a tecnologia de persistencia.

Exemplo de delegacao

```
export function savePatient(
  patient: Patient, userId: string, userInfo?: UserAuditInfo
): Promise<string> {
  return patientService.savePatient(patient, userId, userInfo);
}
```

O codigo de interface chama a funcao de dominio; a fachada delega para o adaptador configurado. A implementacao Firebase implementa IPatientService e concentra as chamadas ao Realtime Database.

Fase 3 ? Testes e automacao

Cobertura funcional adicionada

- **Componentes criticos:** testes de login, etapa de informacoes da dose e painel do dashboard em src/test.
- **Dominio:** testes para formatadores do dashboard, tipos de historico, wrappers de API e modulos de dominio ja cobertos no historico da branch.
- **Servicos:** testes adicionais para activity log, appointments, notes e user settings; os casos incluem operacoes normais, resultados vazios e falhas simuladas.

Exemplo de teste de interface

```
fireEvent.change(screen.getByLabelText("Email"), {
  target: { value: "pessoa@example.com" },
});
fireEvent.change(screen.getByLabelText("Senha"), {
  target: { value: "senha-segura" },
});
expect(aoMudarEmail).toHaveBeenCalledWith("pessoa@example.com");
expect(aoMudarSenha).toHaveBeenCalledWith("senha-segura");
```

O teste verifica que os campos encaminham os valores para os handlers do componente. Outro caso verifica o envio do formulario e o estado desabilitado durante carregamento.

Scripts e CI

```
"test": "vitest run",
"test:coverage": "vitest run --coverage ...",
"build:vercel": "npm test && npm run build"
```

O workflow .github/workflows/ci.yml instala dependencias com npm ci, executa npm test e depois npm run build em pull requests e pushes para main. A configuracao Vercel aponta o build para npm run build:vercel, rodando a suite antes do empacotamento.

Evidencia registrada

Na validacao mais recente: 20 arquivos de teste passaram, totalizando 75 testes; o build de producao terminou com sucesso. ESLint direcionado aos arquivos refatorados, npx tsc --noEmit e git diff --check tambem passaram. O checklist da branch registra a meta de cobertura acordada e a configuracao de CI; os resultados acima sao a execucao observada no build:vercel.

Fase 4 ? Refatoracao das paginas

A divisao preservou o fluxo existente e moveu estado/logica para hooks e blocos de interface para componentes. As paginas foram reduzidas para atuar principalmente como composicao.

Pagina	Linhas antes ? depois	Commit
Appointments	1190 ? 108	2638abf
Medications	964 ? 174	66fa8e7
Clinic	906 ? 83	e75c981
Patients	827 ? 73	63978a0
Settings	784 ? 90	60ef041
Reports	782 ? 113	865c72f
Analytics	546 ? 51	cfc7e4d

Exemplos das extracoes

- **Agenda:** hook useAppointmentSchedule, componentes AppointmentCalendar, AppointmentControls e AppointmentDayPanel; regra de calendario em appointmentCalendar.ts.
- **Pacientes:** hook usePatients e componentes PatientDirectory, PatientFormDialog e PatientViewDialog.
- **Configuracoes:** hook useSettings e paineis separados de Perfil, Seguranca, Notificacoes e IA.
- **Relatorios:** hook useReports, componentes de filtros/previa/estatisticas e geradores PDF/CSV em reportExports.ts.
- **Analytics:** hook useAnalytics e componentes de cabecalho, filtros, estatisticas e dashboard analitico. Clinic e Medications tambem tiveram hooks e componentes extraidos.

Exemplo de composicao

```
<PatientDirectory
  patients={patients.patients}
  filteredPatients={patients.filteredPatients}
  loading={patients.loading}
  searchTerm={patients.searchTerm}
  onView={patients.handleViewClick}
  onEdit={patients.handleEditClick}
  onDelete={patients.handleDeletePatient}
/>
```

A pagina fornece dados e callbacks; a listagem apresenta a tabela e os controles de busca. Essa separacao reduz a complexidade de cada arquivo sem alterar o contrato visual da tela.

Validacao, limitacoes e historico

Comandos e resultados

```
npm run build:vercel
# npm test: 20 arquivos, 75 testes aprovados
# npm run build: compilacao de producao concluida

npx eslint <arquivos-refatorados>
# sem erros (por pagina)

npx tsc --noEmit
# concluido sem erros

git diff --check
# sem erros de whitespace
```

Validacao visual

O login publico foi visualizado no Browser local somente com valores ficticios de Firebase, suficientes para iniciar a aplicacao, mas nao para autenticar. A tentativa inicial sem variaveis resultou em auth/invalid-api-key. As rotas protegidas e os fluxos com dados reais nao foram validados visualmente; permanecem pendentes.

Commits por fase

- **Fase 2:** d226b20 ? contratos, servicos Firebase e migracao da camada de dominio.
- **Fase 3:** fe33633 ? suites de testes de dominio/servicos; fc11aaf ? testes adicionais, scripts, dependencias de teste, CI e build Vercel.
- **Fase 4:** 2638abf, 66fa8e7, e75c981, 63978a0, 60ef041, 865c72f e cfc7e4d ? um commit por pagina refatorada.
- **Estado:** fc11aaf e os demais commits estao no historico local da branch; nao foram enviados ao remoto.

Limites deste relatorio

Este PDF documenta alteracoes e evidencias visiveis no historico local, nao substitui revisao de producao nem validacao manual com ambiente Firebase de teste. A execucao local nao deve ser interpretada como teste de deploy ou verificacao de dados clinicos reais.